StockPot

Sistema web de gestión para restaurantes

Entrega parcial del 70 % — 2.º corte

Ingeniería del Software II
Universidad Simón Bolívar — sede Cúcuta

Docente
Prof. Nataly Duque Páez

Cliente
Sexto Sentido Café & Brunch

20 de octubre de 2026

Equipo

Ángel David Rodríguez Vega
Líder del proyecto (PM)

Andrés Mateo Puentes
Desarrollador

Jhorman Caballero
Desarrollador

Yovany Peñaranda
Desarrollador

Agenda

1. **El proyecto:** problema, objetivos y alcance
2. **El producto:** arquitectura, datos, seguridad y módulos
3. **La gestión:** metodología, Jira, cronograma y estimación
4. **La calidad:** pruebas, estándares y cifras
5. **El cierre:** lecciones, lo que sigue y demostración en vivo

El problema y nuestra solución

Hoy en el restaurante

- Inventario, pedidos y ventas en papel o en hojas de cálculo.
- Se pierden insumos y se compra de más.
- No se conoce el costo real de cada plato.
- Los domicilios y pagos se anotan a mano.
- No hay datos confiables para decidir.

StockPot lo une todo

- **Inventario** con alertas de stock mínimo
- **Recetas** que calculan el costo de cada plato
- **Domicilios y pagos** con estados y comprobante
- **Ventas** que descuentan inventario solas
- **Reportes** para tomar decisiones

Objetivos

Objetivo general: desarrollar un sistema web que permita a los restaurantes pequeños y medianos de Cúcuta gestionar su inventario, compras, menú, pedidos, domicilios, pagos, ventas y reportes en un solo lugar.

- **Controlar el inventario** con entradas, salidas, stock mínimo y alertas.
- **Calcular el costo de cada plato** a partir de su receta.
- **Registrar domicilios, pagos y ventas** y descontar los insumos consumidos.
- **Generar reportes y estadísticas** para la toma de decisiones.
- **Gestionar el proyecto con estándares** (IEEE 1058) y asegurar la calidad con pruebas.

Los 11 módulos y su avance

1. Usuarios y roles	21 PF	Listo
2. Proveedores y compras	34 PF	Listo
3. Inventario	36 PF	Listo
4. Productos y menú	23 PF	Listo
5. Pedidos en mesa	42 PF	Pendiente
6. Cocina	26 PF	Pendiente
7. Reservas	29 PF	Pendiente
8. Domicilios	33 PF	Listo
9. Pagos	29 PF	Listo
10. Ventas y consumo	26 PF	Listo
11. Reportes y estadísticas	31 PF	Listo

Total: 330 PF · 233 entregados

Alcance: 70 % entregado

233 / 330	8 / 11	56 / 80
puntos de función (70,6 %)	módulos funcionando	requisitos funcionales

Avance del software en puntos de función



- **Pendiente:** pedidos en mesa, cocina y reservas, para la entrega final del 13/11/2026.

Arquitectura y tecnología



HTTP + JSON → validación y reglas de negocio → SQL transaccional

Servidor	Acceso remoto	Versiones	Pruebas
Waitress (producción)	enlace público ngrok	Git + GitHub (GitFlow)	pytest + SQLite en memoria

Base de datos: 19 tablas en MySQL 8.4

Seguridad	Compras e inventario	Menú	Pedidos, pagos y ventas
roles	proveedores	productos	clientes
permisos	proveedor_insumos	recetas	ordenes
rol_permisos	ordenes_compra		detalle_ordenes
usuarios	detalle_orden_compra		domicilios
	insumos		pagos
	movimientos_inventario		ventas
			consumos

- **Integridad:** llaves foráneas, restricciones CHECK (nunca stock negativo) y transacciones.
- **Trazabilidad:** cada cambio de stock deja un movimiento; nada se borra físicamente.

Seguridad: 6 roles y 23 permisos

Administrador
Acceso total al sistema

Cajero
Pagos, ventas y domicilios

Bodeguero
Inventario, proveedores y compras

Mesero
Pedidos en mesa y reservas

Cocinero
Órdenes en cocina

Domiciliario
Sus entregas a domicilio

- **Inicio de sesión con token JWT** (vence a las 8 horas).
- **Contraseñas cifradas con scrypt:** nunca se guardan ni se muestran en texto plano.
- **Respuestas claras:** 401 sin sesión, 403 sin permiso, 422 datos inválidos.
- **Protección de la interfaz:** los textos se muestran sin ejecutar código (contra XSS).

Lo que hace el sistema (1/2)

Usuarios y roles

- Registrar, consultar y editar usuarios
- Inactivar sin borrar
- Asignar roles y permisos

Proveedores y compras

- Proveedores e insumos que venden
- Órdenes de compra con estados
- Recibir la compra suma al inventario

Inventario

- Entradas y salidas con motivo
- Stock mínimo, costo y valor
- Alertas de stock bajo o agotado

Productos y menú

- Platos por categoría y precio
- Receta con sus insumos
- Costo, margen y porciones posibles

Lo que hace el sistema (2/2)

Domicilios

- Cliente, dirección y referencia
- Valida que alcance el inventario
- Asignar domiciliario y seguir estados

Pagos

- Efectivo, tarjeta o transferencia
- Pagos parciales y cálculo del cambio
- Comprobante imprimible

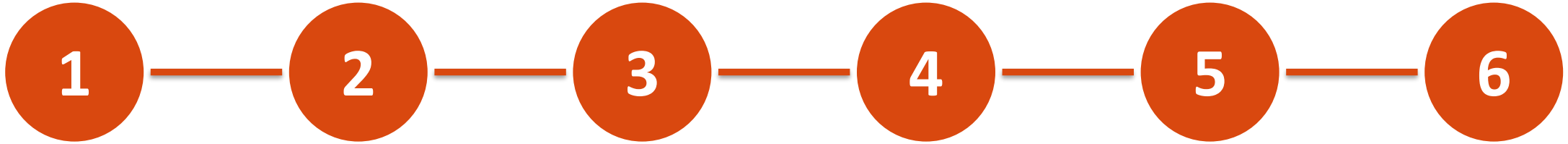
Ventas y consumo

- Venta en caja o al pagar el domicilio
- Descuenta insumos según la receta
- Productos más vendidos

Reportes y estadísticas

- Inventario, ventas, consumo e histórico
- Gráficas y filtro por fechas
- Exportar a Excel (CSV)

Cómo funciona: del insumo al reporte



1	2	3	4	5	6
Compra	**Receta**	**Pedido**	**Pago**	**Venta**	**Reportes**
El insumo entra al inventario	Cada plato sabe qué insumos usa	A domicilio o en caja	Efectivo, tarjeta o transferencia	Descuenta los insumos automáticamente	Información para decidir

Todo en una sola transacción: si falta un insumo, no queda ni el pago ni la venta a medias.

Metodología y gestión

Jira: 166 tareas

123	21	22
listas	en curso	por hacer

- **4 iteraciones,** una por integrante, con un sprint cada una.
- **Fases por iteración:** análisis, diseño, codificación y pruebas.
- **Cronograma:** 10/08 al 13/11/2026 en GanttProject.

GitFlow en GitHub

main	versiones entregadas (v0.7)
develop	integración del equipo
feature/RF-xx	una rama por requisito
docs/…	documentación

78 commits · 32 integraciones
Solo se integra si todas las pruebas pasan.

Estimación: puntos de función y PERT

Puntos de función

- **80** requisitos funcionales analizados
- **330** PF transaccionales (entradas, salidas y consultas)
- **515** PF sin ajustar con archivos lógicos
- **233 PF** entregados en este corte (70,6 %)

PERT

- **Tres tiempos** por tarea: optimista, probable y pesimista
- **Duración esperada:** te = (O + 4M + P) / 6
- **166 tareas** con predecesoras y responsable
- **Predecesoras** para armar el cronograma hasta el 13/11

¿Por qué puntos de función? Miden el tamaño por lo que el usuario recibe, no por líneas de código; así el 70 % se calcula de forma objetiva.

Calidad: pruebas en varios niveles

Aceptación: 10 de 10 criterios cumplidos

Funcionales CRUD: 33 casos en la interfaz

Integración (API): 226 pruebas automáticas

Unitarias: 41 pruebas con pytest

95 %

de cobertura

9

defectos corregidos

267 pruebas automáticas aprobadas (100 %) en cada integración

Estándares aplicados

IEEE 1058	Plan de gestión del proyecto: la entrega está organizada con su estructura.
IEEE 830	Especificación de requisitos: 80 requisitos funcionales (RF-01 a RF-11).
IEEE 1016	Diseño del software: arquitectura en capas y modelo de datos.
IEEE 829 / ISO 29119	Documentación de pruebas: plan, casos, matriz e informe.
ISO/IEC 25010	Calidad del producto: funcionalidad, seguridad y rendimiento.

El proyecto en cifras

≈ 3 300

líneas del backend

≈ 2 800

líneas de pruebas

≈ 5 400

líneas de la interfaz

12

pantallas web

≈ 61

rutas de la API

19

tablas en la base

Lecciones aprendidas y riesgos

Lecciones aprendidas

- Revisar el código antes de probar ahorra correcciones.
- Las reglas críticas van en un solo lugar (inventario).
- Scripts de base de datos que actualizan sin borrar datos.
- Probar también en celular desde el inicio.
- Integrar seguido evita conflictos grandes.

Riesgos y cómo se manejan

- **Atraso de la Iteración 3:** se replanificó sin mover el 13/11.
- **Pérdida de datos:** respaldos en GitHub y scripts incrementales.
- **Errores al integrar:** pruebas automáticas obligatorias.
- **Carga de usuarios:** pruebas de carga pendientes.

Lo que sigue: el 30 % restante

1

Iteración 3
Pedidos en mesa, cocina y reservas

21 de octubre al 13 de noviembre

2

Pruebas de aceptación
Con el cliente Sexto Sentido Café & Brunch

Antes de la entrega final

3

Entrega final y sustentación
100 % del software y documentación

13 de noviembre de 2026

Demostración en vivo

Del registro del usuario al reporte de ventas, con el sistema funcionando.

¡Gracias!

¿Preguntas?

Código: github.com/Angelssjyf/stockpot